

ECE220. ΑΡΙΘΜΗΤΙΚΗ ΑΝΑΛΥΣΗ

ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΙΑΣ
ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧ. & ΜΗΧ. ΥΠΟΛΟΓΙΣΤΩΝ

Πίνακες, Βρόχοι, Εντολές Διακλάδωσης

Πίνακες (Arrays) – Δημιουργία ειδικών πινάκων

- ▶ Πίνακας τυχαίων **πραγματικών** αριθμών στο διάστημα $(0,1)$

`A = rand(n,m)` πίνακας $n \times m$

`A = rand(n)` πίνακας $n \times n$

```
>>rand(2,3)
```

```
ans =
```

0.8147	0.1270	0.6324
0.9058	0.9134	0.0975

- ▶ Πίνακας τυχαίων **ακέραιων** αριθμών στο διάστημα $[a, b]$

`A = randi([a,b],n,m)` πίνακας $n \times m$

`A = randi([a,b],n)` πίνακας $n \times n$

```
>>randi([3,40],2,3)
```

```
ans =
```

39	33	19
21	8	37

Πίνακες (Arrays) – Δημιουργία ειδικών πινάκων

► Μοναδιαίος πίνακας

$A = \text{eye}(n)$ μοναδιαίος πίνακας $n \times n$

$A = \text{eye}(n, m)$ μοναδιαίος πίνακας $n \times m$ (μη τετραγωνικός)

`eye(3)`

`ans =`

1	0	0
0	1	0
0	0	1

`eye(3,4)`

`ans =`

1	0	0	0
0	1	0	0
0	0	1	0

Πίνακες (Arrays) – Δημιουργία ειδικών πινάκων

► Διαγώνιος πίνακας

$A = \text{diag}(V)$ διαγώνιος πίνακας $n \times n$ τα στοιχεία του V τοποθετούνται στην κύρια διαγώνιο

$A = \text{diag}(V, i)$ διαγώνιος πίνακας $m \times m$ τα στοιχεία του V τοποθετούνται στην i – οστή διαγώνιο

```
>>diag ([1 2 3 4])
```

ans =

1	0	0	0
0	2	0	0
0	0	3	0
0	0	0	4

```
>>diag ([1 2 3 4],1)
```

ans =

0	1	0	0	0
0	0	2	0	0
0	0	0	3	0
0	0	0	0	4
0	0	0	0	0

```
>>diag ([1 2 3 4],-2)
```

ans =

0	0	0	0	0	0
0	0	0	0	0	0
1	0	0	0	0	0
0	2	0	0	0	0
0	0	3	0	0	0
0	0	0	4	0	0

Συναρτήσεις πινάκων

► Συνάρτηση

- `tril(A)` – επιστρέφει το κάτω τριγωνικό τμήμα ενός πίνακα A
- `tril(A,k)` – επιστρέφει τα στοιχεία του A που βρίσκονται στη k -στη διαγώνιο και κάτω από αυτή.
- `triu(A)` – επιστρέφει το άνω τριγωνικό τμήμα ενός πίνακα A
- `istril(A)` – ελέγχει αν ο πίνακας A είναι κάτω τριγωνικός
- `magic(n)` – δημιουργεί $n \times n$ πίνακα ακεραίων με τιμές από 1 μέχρι n^2 τέτοιο ώστε, τα αθροίσματα των γραμμών και των στηλών του να είναι ίσα.

```
>> A=magic(3)
```

```
A =
```

8	1	6
3	5	7
4	9	2

```
>> tril(A)
```

```
ans =
```

8	0	0
3	5	0
4	9	2

```
>> triu(A)
```

```
ans =
```

8	1	6
0	5	7
0	0	2

Άσκηση 1

- ▶ Να γραφεί κώδικας ο οποίος
 - a. Δημιουργεί ένα πίνακα A 4×5 τυχαίων ακεραίων στον διάστημα $[0, 10]$
 - b. Αποθηκεύει σε ένα πίνακα B την 1η και 3η γραμμή του A
 - c. Αποθηκεύει σε ένα πίνακα C την 2η και 4η στήλη του A

Άσκηση 1 – Λύση

- ▶ Να γραφεί κώδικας ο οποίος
 - a. Δημιουργεί ένα πίνακα A 4×5 τυχαίων ακεραίων στον διάστημα $[0, 10]$
 - b. Αποθηκεύει σε ένα πίνακα B την 1η και 3η γραμμή του A
 - c. Αποθηκεύει σε ένα πίνακα C την 2η και 4η στήλη του A

```
>> A=randi([0,10],4,5)
```

A =

10	8	4	3	1
0	8	7	7	5
4	2	7	7	10
4	5	8	1	3

```
>> B=[A(1,:); A(3,:)]
```

B =

10	8	4	3	1
4	2	7	7	10

```
>> C=[A(:,2) , A(:,4)]
```

C =

8	3
8	7
2	7
5	1

Άσκηση 2

- ▶ Να γραφεί κώδικας ο οποίος
 - a. Δημιουργεί ένα πίνακα A 3×3 τυχαίων ακεραίων στον διάστημα $[0, 10]$
 - b. Δημιουργεί τον μοναδιαίο πίνακα I 3×3 και τον αποθηκεύει στη μεταβλητή B
 - c. Υπολογίζει του πίνακες $C = A + B$, $D = B/2$ και $E = A - 3B$

Άσκηση 2 – Λύση

- ▶ Να γραφεί κώδικας ο οποίος
 - a. Δημιουργεί ένα πίνακα A 3×3 τυχαίων ακεραίων στον διάστημα $[0, 5]$
 - b. Δημιουργεί τον μοναδιαίο πίνακα I 3×3 και τον αποθηκεύει στη μεταβλητή B
 - c. Υπολογίζει του πίνακες $C = A + B$, $D = B/2$ και $E = A - B$

`A = randi([0,5],3,3)`

$A =$

3	1	5
1	3	5
4	4	3

`B = eye(3)`

$B =$

1	0	0
0	1	0
0	0	1

`C=A+B`

$C =$

4	1	5
1	4	5
4	4	4

`D=B/2`

$D =$

0.5	0	0
0	0.5	0
0	0	0.5

`E=A-3*B`

$E =$

0	1	5
1	0	5
4	4	0

Γινόμενο πινάκων

- ▶ Γινόμενο πινάκων $A * B$
- ▶ Ο αριθμός των στηλών του A θα πρέπει να είναι ίσος με το αριθμό των γραμμών του B

```
>> A = [1 2 3; 4 5 6]
```

A =

1	2	3
4	5	6

```
>> B = [ 1 2; 3 4; 5 6]
```

B =

1	2
3	4
5	6

```
>> A*B
```

ans =

22	28
49	64

Συναρτήσεις πινάκων - Αντίστροφος πίνακας

- ▶ Ένας τετραγωνικός πίνακας A ονομάζεται αντιστρέψιμος όταν υπάρχει πίνακας B τ.ω.
 - ▶ $A * B = B * A = I$
- ▶ Ο B ονομάζεται αντίστροφος του A και συμβολίζεται A^{-1}
- ▶ Ο A είναι αντιστρέψιμος αν $\det(A) \neq 0$

```
>> A=[1 1 1; 2 2 3; 5 6 7]
```

```
A =
```

```
1    1    1
2    2    3
5    6    7
```

```
>> inv(A)
```

```
ans =
```

```
4.0000    1.0000   -1.0000
-1.0000   -2.0000    1.0000
-2.0000    1.0000     0
```

Ορίζουσα πινάκα

- ▶ Ορίζουσα πίνακα A $n \times n$ $\det(A)$

```
>>A
```

```
A =
```

```
1 -2 4  
-5 2 0  
1 0 5
```

```
>>det(A)
```

```
ans =
```

```
-48
```

$$\begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$$

$$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = a \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix} = aei + bfg + cdh - ceg - bdi - afh$$

Γραμμικά Συστήματα - $x = \text{inv}(A) * b$

- ▶ Γραμμικό Σύστημα $A * x = b$
- ▶ Επίλυση γραμμικού συστήματος με MATLAB
- ▶ $x = \text{inv}(A) * b$
 - ▶ *Μπορεί να χρησιμοποιηθεί στην περίπτωση που ο πίνακας A είναι αντιστρέψιμος.*
 - ▶ *Οι στήλες ή οι γραμμές του A πρέπει να είναι γραμμικά ανεξάρτητες.*
 - ▶ *Καλώς ορισμένο σύστημα (δηλ. A τετραγωνικός).*

Δομές ελέγχου – `if`

► Επιλογή

```
if λογική έκφραση  
    εντολές  
end
```

► Σύνθετη επιλογή

```
if λογική έκφραση  
    εντολές 1  
else  
    εντολές 2  
end
```

► Πολλαπλή επιλογή

```
if λογική έκφραση 1  
    εντολές 1  
else if λογική έκφραση 2  
    εντολές 2  
else  
    εντολές 3  
end
```

Δομές ελέγχου - **if** παράδειγμα

```
A=rand(3,4);  
[m, n] = size(A);  
if m ~= n  
    display('Matrix is not square.')  
end
```

```
x = input('Insert a number:');  
  
if x ~= 10  
    disp('x is not 10')  
else  
    disp('x is 10')  
end
```


Δομές ελέγχου – switch

```
switch έκφραση
    case έκφραση
        εντολές
    case έκφραση
        εντολές
    ...
    otherwise
        εντολές
end
```

```
n = input('Enter a number: ');

switch n
    case -1
        disp('negative one')
    case 0
        disp('zero')
    case 1
        disp('positive one')
    case {2,3}
        disp('two or three')
    otherwise
        disp('other value')
end
```

Δομές επανάληψης

- ▶ Απαραίτητες όταν απαιτείται επανάληψη μίας ή περισσότερων ενεργειών
- ▶ MATLAB διαθέτει δύο δομές επανάληψης
 - ▶ `for`
 - ▶ `while`

Δομή επανάληψης

for

- ▶ Εκτελεί ένα σύνολο εντολών σε ένα βρόχο συγκεκριμένο αριθμό επαναλήψεων.

```
for index = αρχική_τιμή:τελική_τιμή  
    εντολές  
end
```

```
for index = αρχική_τιμή:βήμα:τελική_τιμή  
    εντολές  
end
```

```
for index = διάνυσμα_ή_πίνακας  
    εντολές  
end
```

```
>> for v = 1:2:5  
    disp(v)  
end  
  
1  
  
3  
  
5
```

```
>> for v = [1 5 8 17]  
    disp(v)  
end  
  
1  
  
5  
  
8  
  
17
```

Δομή επανάληψης

while

- ▶ Εκτελείται όσο η έκφραση είναι αληθής

```
while έκφραση  
    εντολές  
end
```

```
n = 10;  
while n > 1  
    disp('hello')  
    n = n-1  
end
```

Διανυσματοποίηση

- Στο παρελθόν η χρήση βρόχων επανάληψης (`for` και `while`) στο MATLAB μπορούσε να οδηγήσει σε μεγάλους χρόνους εκτέλεσης.
- Οι πράξεις με διανύσματα και τμήματα πινάκων (matrices) ήταν πολύ αποδοτικότερες.
- Στις τελευταίες εκδόσεις του MATLAB η διαφορά αυτή έχει μειωθεί, λόγω βελτιστοποιήσεων που έχουν γίνει στην μηχανή εκτέλεσης.
- Η τεχνική αντικατάστασης βρόχων, με πράξεις μεταξύ πινάκων αναφέρεται ως διανυσματοποίηση ή vectorization.

Άνω τριγωνικό σύστημα – Επίλυση κατά γραμμές

Επίλυση άνω τριγωνικού συστήματος

Παράδειγμα

$$x_n = b_n / u_{nn}$$

για $k = n-1, n-2, \dots, 1$

$$x_k = \frac{b_k - \sum_{j=k+1}^n u_{kj} x_j}{u_{kk}}$$

$$2x_1 - 2x_2 + x_3 = 6$$

$$x_2 + 2x_3 = 3$$

$$-17.5x_3 = -35$$

$$U = \begin{bmatrix} 2 & -2 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & -17.5 \end{bmatrix} \quad b = \begin{bmatrix} 6 \\ 3 \\ -35 \end{bmatrix} \quad x = \begin{bmatrix} \\ \\ \end{bmatrix}$$

$$n = 3$$

$$x_3 = b_3 / U_{33}$$

$$-17.5x_3 = -35 \Leftrightarrow x_3 = 2$$

$$k = 2$$

$$x_2 = (b_2 - U_{23} * x_3) / a_{22}$$

$$x_2 + 2x_3 = 3 \Leftrightarrow x_2 = 3 - 2 \cdot 2 \Leftrightarrow x_2 = -1$$

$$k = 1$$

$$x_1 = (b_1 - (U_{12} * x_2 + U_{13} * x_3)) / a_{11}$$

$$2x_1 - 2x_2 + x_3 = 6 \Leftrightarrow 2x_1 = 6 - (-2 \cdot (-1) + 2) \Leftrightarrow x_1 = 1$$

Άνω τριγωνικό σύστημα – Επίλυση κατά γραμμές

Επίλυση άνω τριγωνικού συστήματος

Παράδειγμα

$$x_n = b_n / u_{nn}$$

για $k = n-1, n-2, \dots, 1$

$$x_k = \frac{b_k - \sum_{j=k+1}^n u_{kj} x_j}{u_{kk}}$$

$n = 3$

$$2x_1 - 2x_2 + x_3 = 6$$

$$x_2 + 2x_3 = 3$$

$$-17.5x_3 = -35$$

$$U = \begin{bmatrix} 2 & -2 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & -17.5 \end{bmatrix} \quad b = \begin{bmatrix} 6 \\ 3 \\ -35 \end{bmatrix} \quad x = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

$$x_3 = b_3 / U_{33}$$

$$-17.5x_3 = -35 \Leftrightarrow x_3 = 2$$

$k = 2$

$$x_2 = (b_2 - U_{23} * x_3) / a_{22}$$

$$x_2 + 2x_3 = 3 \Leftrightarrow x_2 = 3 - 2 \cdot 2 \Leftrightarrow x_2 = -1$$

$k = 1$

$$x_1 = (b_1 - (U_{12} * x_2 + U_{13} * x_3)) / a_{11}$$

$$2x_1 - 2x_2 + x_3 = 6 \Leftrightarrow 2x_1 = 6 - (-2 \cdot (-1) + 2) \Leftrightarrow x_1 = 1$$

Εσωτερικό γινόμενο

$$U(1,2:3) * x(2:3)$$

Για την k επανάληψη

$$U(k, k+1:n) * x(k+1:n)$$

Άνω τριγωνικό σύστημα – Επίλυση κατά γραμμές

Λύση άνω τριγωνικού συστήματος

$$x_n = b_n / u_{nn}$$

για $k = n-1, n-2, \dots, 1$

$$x_k = \frac{b_k - \sum_{j=k+1}^n u_{kj} x_j}{u_{kk}}$$

Υλοποίηση αλγορίθμου

```
x(n) = b(n)/U(n,n);  
for k=n-1:-1:1  
    s = 0;  
    for jj=k+1:1:n  
        s = s + U(k,jj)* x(jj);  
    end  
    x(k) = (b(k) - s)/U(k,k);  
end
```

Υλοποίηση αλγορίθμου με διανυσματοποίηση

```
x(n) = b(n)/U(n,n);  
for k=n-1:-1:1  
    x(k) = (b(k) - U(k,k+1:n)*x(k+1:n))/U(k,k);  
end
```